

MATEMÁTICA COMPUTACIONAL APLICADA

Aula 06

Álgebra Booleana e suas Aplicações — Parte 1

Da tabela-verdade para os axiomas — a mesma lógica, agora manipulada como álgebra.

$x + y$

$x \cdot y$

x^{-}

Nesta aula



Da tabela-verdade para a álgebra sobre $\{0, 1\}$

A mesma estrutura lógica, agora com operações e axiomas.



Axiomas e teoremas fundamentais

As regras que permitem manipular fórmulas sem montar tabela nenhuma.



Simplificação algébrica de expressões

Reduzir o número de operações — menos termos, menos trabalho depois.



Base em Python (e C#) com operadores bit a bit

Confirmando cada simplificação por código, não só no papel.

Onde paramos na Aula 05

Aprendemos a montar tabelas de fórmulas com vários conectivos e a classificá-las.



Tautologia

sempre V



Contradição

sempre F



Contingência

varia



Hoje: a mesma lógica, mas sobre $\{0, 1\}$ e com regras algébricas para manipular fórmulas direto.



Da lógica para a álgebra

Na Aula 04 trabalhamos com V/F. A álgebra booleana é a mesma estrutura, com 1 (verdadeiro) e 0 (falso), e três operações.

Operação	Notação	Equivalente lógico
Produto (AND)	$x \cdot y$ ou xy	$\wedge$
Soma (OR)	$x + y$	$\vee$
Complemento (NOT)	x^- ou x'	$\neg$

Reforçando com uma tabela em 0/1:

x	y	$x \cdot y$	$x + y$	x^-
1	1	1	1	0
1	0	0	1	0
0	1	0	1	1
0	0	0	0	1

ATENÇÃO



Cuidado com os símbolos

+ e · não são a soma e a multiplicação da aritmética comum! Aqui, + significa OR — por isso $1 + 1 = 1$ (não 2).

$$1 + 1 = 1$$

Axiomas e teoremas fundamentais

Cada axioma vem em par: uma forma com + (OR) e uma forma dual com · (AND).

Axioma	Forma com +	Forma com ·
Identidade	$x + 0 = x$	$x \cdot 1 = x$
Complemento	$x + x^{-} = 1$	$x \cdot x^{-} = 0$
Idempotência	$x + x = x$	$x \cdot x = x$
Absorção	$x + xy = x$	$x \cdot (x + y) = x$
Distributiva	$x + yz = (x+y) (x+z)$	$x (y + z) = xy + xz$
De Morgan	$(x + y)^{-} = x^{-} \cdot \bar{y}$	$(xy)^{-} = x^{-} + \bar{y}$

Guarde esta tabela — ela é a base de toda simplificação a partir daqui.



Princípio da dualidade

Toda identidade booleana continua válida se trocarmos $+$ e $\cdot$ (e 0 e 1) ao mesmo tempo.

$$x + 0 = x$$



$$x \cdot 1 = x$$

axioma da Identidade — dual de si mesmo, trocando $+$ $\leftrightarrow$ $\cdot$ e $0 \leftrightarrow 1$

Os axiomas sempre vêm aos pares — por isso a tabela do slide anterior tem duas colunas: aprender um lado é meio caminho para o outro.



Simplificando expressões

Simplificar reduz o número de operações — o que, mais adiante no curso (circuitos), significa menos portas lógicas e hardware mais barato.

EXEMPLO 1

Fatorando um complemento

Simplificar $xy + x\bar{y}$:

1

$$x(y + \bar{y})$$

Fatoramos x , que aparece nos dois termos.

2

$$x \cdot 1$$

Axioma do complemento: $y + \bar{y} = 1$.

3

$$x$$

Axioma da identidade: $x \cdot 1 = x$.

Duas parcelas viraram um único literal.

CONFIRMANDO

$xy + x\bar{y}$ é o mesmo que x

Montamos as 4 combinações de x e y e comparamos as colunas.

x	y	xy	$x\bar{y}$	$xy + x\bar{y}$	x
1	1	1	0	1	1
1	0	0	1	1	1
0	1	0	0	0	0
0	0	0	0	0	0

As duas últimas colunas são idênticas em todas as linhas — a simplificação está correta.

EXEMPLO 2

Usando absorção

Simplificar $x + x^{\sim}y$:

1 $(x + x^{\sim})(x + y)$

Distributiva na forma dual: $x + yz = (x+y)(x+z)$, com $y = \bar{x}$ e $z = y$.

2 $1 \cdot (x + y)$

Axioma do complemento: $x + \bar{x} = 1$.

3 $x + y$

Axioma da identidade: $1 \cdot (x + y) = x + y$.

CONFIRMANDO

$x + \bar{x}y$ é o mesmo que $x + y$

Mesma checagem: 4 linhas, comparar as colunas finais.

x	y	$\bar{x}y$	$x + \bar{x}y$	$x + y$
1	1	0	1	1
1	0	0	1	1
0	1	1	1	1
0	0	0	0	0

Note que $x + \bar{x}y$ nunca é F quando $x + y$ também não é — as fórmulas coincidem sempre.



Eliminando o consenso

Simplifique $xy + \bar{x}z + yz$:

O termo yz é redundante: sempre que $yz = 1$ (ou seja, $y = 1$ e $z = 1$), pelo menos um de xy ou $\bar{x}z$ também vale 1 — porque x só pode ser 0 ou 1, e em qualquer caso um dos dois primeiros termos "cobre" essa situação.

$$xy + \bar{x}z + yz = xy + \bar{x}z$$

Esse é o teorema do consenso: o termo yz — formado pelas variáveis que sobram de xy e $\bar{x}z$ — sempre pode ser eliminado.

CONFIRMANDO POR TABELA-VERDADE

As 8 linhas, lado a lado

Monte $xy + \bar{x}z + yz$ e $xy + \bar{x}z$ e compare a coluna de resultado.

x	y	z	xy	$\bar{x}z$	yz	$xy + \bar{x}z + yz$	$xy + \bar{x}z$
1	1	1	1	0	1	1	1
1	1	0	1	0	0	1	1
1	0	1	0	0	0	0	0
1	0	0	0	0	0	0	0
0	1	1	0	1	1	1	1
0	1	0	0	0	0	0	0
0	0	1	0	1	0	1	1
0	0	0	0	0	0	0	0

As colunas finais são idênticas — o termo de consenso realmente não fazia falta.



Menos termos, menos hardware

Cada operação (+, · ou complemento) que sobrevive na fórmula final vira, mais adiante no curso, uma porta lógica física. $xy + \bar{x}z + yz$ e $xy + \bar{x}z$ fazem exatamente o mesmo trabalho — mas a segunda usa menos peças.

$$xy + \bar{x}z + yz$$

3 ANDs, 2 ORs, 1 NOT



$$xy + \bar{x}z$$

2 ANDs, 1 OR, 1 NOT

um AND e um OR a menos — em um circuito real, isso é peças a menos na placa.

CONFIRMANDO COM CÓDIGO



Álgebra booleana em código

Em vez de confiar só no papel, vamos implementar AND, OR e NOT sobre 0/1 e testar cada simplificação automaticamente.

PASSO 1 · AS TRÊS OPERAÇÕES

AND, OR e NOT sobre 0/1

Em Python e C#, os operadores bit a bit já fazem exatamente o que precisamos.

BooleanaBasica.cs



```
static int And(int x, int y) => x & y;  
static int Or(int x, int y)  => x | y;  
static int Not(int x) => 1 - x;
```

booleana_basica.py



```
def AND(x, y): return x & y  
def OR(x, y):  return x | y  
def NOT(x):    return 1 - x
```

&, | e ~ são operadores bit a bit — funcionam igual nas duas linguagens porque x e y valem só 0 ou 1.

Toda combinação de 0 e 1

Mesma ideia da Aula 05, agora devolvendo inteiros em vez de booleanos.

Combinacoes.cs



```
static IEnumerable<int[]> Combinacoes(int n)
{
    for (int i = 0; i < (1 << n); i++)
    {
        var v = new int[n];
        for (int j = 0; j < n; j++)
            v[j] = (i >> j) & 1;
        yield return v;
    }
}
```

combinacoes.py



```
from itertools import product

for valores in product((0, 1), repeat=n):
    print(valores)
```

Igual à Aula 05: cada número de 0 a 2^n-1 , em binário, já é uma combinação de 0/1.

PASSO 3 · TESTAR A SIMPLIFICAÇÃO

$xy + x\bar{y}$ simplifica para x ?

Comparamos a fórmula original com a simplificada em todas as combinações.

TestandoSimplificacao.cs



```
static int Original(int x, int y) =>
    Or(And(x, y), And(x, Not(y)));

static int Simplificada(int x, int y) => x;

foreach (var x in new[] { 0, 1 })
    foreach (var y in new[] { 0, 1 })
        Debug.Assert(
            Original(x, y) == Simplificada(x, y));
```

testando_simplificacao.py



```
def original(x, y):
    return OR(AND(x, y), AND(x, NOT(y)))

def simplificada(x, y):
    return x

for x in (0, 1):
    for y in (0, 1):
        assert original(x, y) == simplificada(x, y)

print("Simplificação confirmada!")
```

Testando equivalência automaticamente

Uma função que serve para qualquer par de fórmulas, não só para esta.

Equivalentes.cs



```
static bool Equivalentes(  
    Func<int[], int> f,  
    Func<int[], int> g, int n) =>  
    Combinacoes(n).All(v => f(v) == g(v));  
  
Equivalentes(v => Original(v[0], v[1]),  
    v => Simplificada(v[0], v[1]), 2); // True
```

equivalentes.py



```
from itertools import product  
  
def equivalentes(f, g, n):  
    return all(f(*b) == g(*b) for b in product((0, 1),  
        repeat=n))  
  
print(equivalentes(original, simplificada, 2)) # True
```



Hora de praticar

Quatro exercícios: tabela-verdade, simplificação, De Morgan e um desafio com NAND/NOR.



EXERCÍCIO 1

Tabela-verdade

Monte a tabela-verdade de $\bar{x} + xy$ (2 variáveis, 4 linhas). Compare com a tabela de $p \rightarrow q$ da Aula 04 — o que você percebe?

x	y	xy	$\bar{x} + xy$
1	1	?	?
1	0	?	?
0	1	?	?
0	0	?	?



GABARITO · EXERCÍCIO 1

Tabela-verdade

x	y	xy	$x^- + xy$
1	1	1	1
1	0	0	1
0	1	0	1
0	0	0	0

$\bar{x} + xy$ dá 1,0,1,1 — exatamente a tabela de $p \rightarrow q$ da Aula 04. Faz sentido: por distributiva dual, $\bar{x}+xy = (\bar{x}+x)(\bar{x}+y) = \bar{x}+y$, e $\bar{x}+y$ é a forma booleana de $\neg p \vee q$.



EXERCÍCIO 2

Simplificando

Simplifique $xy + x\bar{y} + \bar{x}y$ usando os teoremas. Dica: agrupe os dois primeiros termos primeiro.

$$xy + x\bar{y} + \bar{x}y$$

Passo a passo (use os axiomas da tabela mestra):



GABARITO · EXERCÍCIO 2

Simplificando

1

$$(xy + x\bar{y}) + x^{-}y$$

Agrupamos os dois primeiros termos, como sugerido.

2

$$x + x^{-}y$$

$xy + x\bar{y} = x$, pelo exemplo "Fatorando um complemento".

3

$$x + y$$

$x + \bar{x}y = x + y$, pelo exemplo "Usando absorção".

Resultado: $xy + x\bar{y} + x^{-}y = x + y$



EXERCÍCIO 3

Aplicando De Morgan

Use De Morgan para reescrever a expressão abaixo sem barra sobre grupos — só sobre literais isolados.

$$(xy + z)^{-}$$

Passo a passo (dica: vai precisar aplicar De Morgan duas vezes):



GABARITO · EXERCÍCIO 3

Aplicando De Morgan

1 $(xy)^- \cdot z^-$

De Morgan na soma externa: $(A + z)^- = \bar{A} \cdot \bar{z}$, com $A = xy$.

2 $(x^- + \bar{y}) \cdot z^-$

De Morgan de novo, agora em $(xy)^- = \bar{x} + \bar{y}$. Bar só sobre literais.

Resultado: $(xy + z)^- = (x^- + \bar{y}) \cdot z^-$



EXERCÍCIO 4 · DESAFIO

NAND, NOR e De Morgan

Implemente $\text{NAND}(x, y)$ e $\text{NOR}(x, y)$ em Python (e C#) e use equivalentes / Equivalentes para confirmar que $\text{NAND}(x, y) = \bar{x} + \bar{y}$ e $\text{NOR}(x, y) = \bar{x} \cdot \bar{y}$.

dica: $\text{NAND} = \text{NOT}(\text{AND}(\dots))$ e $\text{NOR} = \text{NOT}(\text{OR}(\dots))$ — os nomes já dizem o que fazer

● ● ● nand_nor.py

```
def NAND(x, y): return _____
```

```
def NOR(x, y): return _____
```

Teste: `equivalentes(lambda x,y: NAND(x,y), lambda x,y: OR(NOT(x),NOT(y)), 2)` deve dar True.



GABARITO · EXERCÍCIO 4

NAND, NOR e De Morgan

nand_nor.py



```
def NAND(x, y): return NOT(AND(x, y))
```

```
def NOR(x, y): return NOT(OR(x, y))
```

Equivalente em C#: `static int Nand(int x, int y) => Not(And(x, y));` (e Nor análogo)

x	y	NAND	$\bar{x} + \bar{y}$
1	1	0	0
1	0	1	1
0	1	1	1
0	0	1	1

Por De Morgan, $NAND(x,y) = (xy)^{\bar{}} = \bar{x} + \bar{y}$ e, de forma análoga, $NOR(x,y) = (x+y)^{\bar{}} = \bar{x} \cdot \bar{y}$.

Onde estamos no semestre



Depois dos axiomas e da simplificação, a Aula 07 mostra onde tudo isso vira circuito de verdade.

Referências

Livros — teoria



ROSEN, K. H.

Matemática Discreta e suas Aplicações. 7. ed.
AMGH/McGraw-Hill — cap. Álgebra Booleana.



GERSTING, J. L.

Fundamentos Matemáticos para a Ciência da
Computação. LTC — cap. Álgebra de Boole e
Computação.



IDOETA, I. V.; CAPUANO, F. G.

Elementos de Eletrônica Digital. Érica — cap.
Álgebra de Boole e simplificação.

Documentação e vídeos



Python — operadores bit a bit

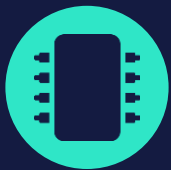
[&, | e ~ \(bitwise operations\)](#) — a base de AND, OR e NOT sobre 0/1.



Vídeos

[Álgebra booleana — axiomas e simplificação](#) — passo a passo.

[Teorema do consenso](#) — simplificação booleana avançada.



PRÓXIMA PARADA

Álgebra Booleana e suas Aplicações — Parte 2

Dos axiomas no papel para portas lógicas de verdade.

Guarde a tabela de axiomas e o teorema do consenso — é exatamente o que vamos usar para desenhar circuitos com menos peças.